

IN-CLASS ACTIVITY SHEET

Cross-Platform Mobile App Development & Testing

Case Study: “QuickBite” – Campus Food Ordering App

Note: Any established cross-platform framework may be used for this activity — Flutter (Dart) or React Native (JavaScript/TypeScript). Choose the framework you are most comfortable with, or the one assigned by your instructor. Screens, tasks, and rubric below apply equally to either choice.

Background: The university canteen wants a mobile app that lets students browse the daily menu, order food ahead of time, and track their order status, reducing queue time between lectures. You have been hired as the developer team to design and build a working prototype (MVP) of this app, running on both Android and iOS from a single codebase.

Functional Requirements

- Splash screen and simple Login/Guest-access screen.
- Home screen listing menu items by category (e.g., Meals, Beverages, Snacks) with images, price and a search/filter bar.
- Item detail view with an “Add to Cart” action and quantity selector.
- Cart screen showing selected items, quantities, subtotal, and a Checkout button.
- Order confirmation screen showing an order number and estimated pickup time.
- Order tracking / status screen (e.g., Placed → Preparing → Ready for pickup).
- Basic user profile screen (name, order history).

Non-Functional Requirements

- Must run on both Android and iOS from one shared codebase.
- Responsive layout that adapts to different screen sizes.
- Smooth navigation between screens (max 1–2 seconds transition).
- Cart state must persist while navigating between screens.

Tools & Technology Stack

Category	Suggested Tools
Framework	Flutter (Dart) OR React Native (Expo / JavaScript-TypeScript) — pick one
IDE / Editor	Visual Studio Code / Android Studio
Emulator / Device	Android Emulator, iOS Simulator (Mac), or a physical phone via USB debugging / Expo Go
Version Control	Git & GitHub / GitLab repository for the project
Testing	Manual functional testing; optional: Flutter widget tests or Jest + React Native Testing Library

Activity Tasks

Part A – Environment Setup (15–20 min)

1. Install the SDK for your chosen framework (Flutter SDK, or Node.js + Expo CLI for React Native).
2. Verify the setup (“flutter doctor” or “npx expo --version”) and start an emulator or connect a device.
3. Create a new project named quickbite_app and confirm the default starter screen runs successfully.

Part B – Build the UI Screens (60–90 min)

4. Create the screen/route structure: Splash → Login → Home → Item Detail → Cart → Checkout → Order Tracking → Profile.
5. Build the Home screen: a scrollable/grid list of menu items pulled from a local list or JSON file (image, name, price).
6. Implement navigation between screens using the framework's navigation library (Navigator / go_router for Flutter, React Navigation for React Native).
7. Implement “Add to Cart” logic and a Cart screen that reads from shared app state.
8. Style the app with a consistent theme (colors, fonts) and make sure layouts are responsive.

Part C – State, Logic & Data (30–45 min)

9. Implement cart state management (e.g., Provider/Riverpod/useState in Flutter, or Context API/Redux in React Native).
10. Calculate and display the cart subtotal and item count dynamically.
11. Simulate order placement: generate an order ID and move the order through at least two status states.

Part D – Build & Test (30–45 min)

12. Run the app on an emulator/device for both a phone-size and tablet-size screen (or resize the emulator window).
13. Design at least 6 test cases covering navigation, cart logic, form/input validation, and layout responsiveness.
14. Execute each test case manually, record Pass/Fail, and capture a screenshot as evidence.
15. Fix any bugs discovered and re-test until all critical test cases pass.

Deliverables & Submission

- A completed Submission Report, containing app screenshots, key code snippets with explanation, test log and the GitHub Repo Link.
- Submit the report through the LMS / designated folder before the deadline.